PHYS30762 Object Oriented Programming in C++

Analogue Circuits Report

Kuba Kane
11019699

Department of Physics and Astronomy
The University of Manchester

May 2026

Abstract

This report explains the considerations taken when building a program which allows the construction of analogue circuits made up of resistors, capacitors and inductors. The program is built on the paradigm of object oriented programming, using classes to represent the three possible components and a generic series or parallel circuit. Inheritance is used maximally, where all components and the circuit classes are inherited from an abstract base class for a generic component, allowing for circuits themselves to be added as components to different circuits. All classes are abstracted in the sense that they have implementation files, apart from simple structs and enumerator classes which are used by one class only. This program provides a way of building a *realistic* circuit, enforcing components to be in ranges of real resistors, capacitors and inductors as specified by validation files and functions. Such functions have been kept as private or protected, ensuring that a program user is unable to bypass checks and build a circuit with unrealistic components. The introduction of IDs and counters have enabled components within a circuit to be kept a track of efficiently, and function overriding has allowed the calculation of impedance for all components and the circuit without code repetition.

Contents

1 Introduction **2**

2 Code design and implementation **2**

3 Results **6**

4 Discussion and conclusions **7**

References **8**

1. Introduction

This program allows users to build analogue circuits using resistors, capacitors, and inductors (which are all classes derived from an abstract base `Component` class). A `Circuit` class can be set to be series or parallel, and any number of components can be added to it, which is then stores as a vector of components (specifically a vector of unique pointers to components). Since the `Circuit` class is inherited from the `Component` class itself, it is possible to add series circuits to parallel circuits, or vice versa, and therefore construct circuits of any topology.

In the main file of this program, a *library* of components is created, which stores different types of resistors, capacitors and inductors. These components are validated through the use of validation files, such that if the user tries to initialise a component with properties which do not match those specified in the validation file, an error message will be returned. The validation files specify a component name, like *Wirewound* for resistor, as well as a tolerance to dissipative effects, and a maximum frequency to which they can withstand [4]. Each validation file then contains maximum and minimum possible values for the resistance, capacitance and inductance respectively.

The `Component` base class contains an operating frequency `frequency` and a unique ID `component_id`, as well as a function to calculate the impedance of the component (using the complex library in C++), and its magnitude and phase difference (`get_impedance()`, `get_impedance_magnitude`, `get_phase_difference`). It also has a `base_info()` function which outputs these properties, and which is overriden in the derived classes to print the resistance, capacitance and inductance of the corresponding classes. A summary of the data members and member functions can be seen in figure 1.

This program therefore gives the user an easy way of constructing circuits with nested branches and varying topologies, as well as ensuring at the same time that the components are physically meaningful, much like an undergraduate circuit labaratory where only certain types of components are available and can be used to build circuits.

2. Code design and implementation

We continue with this idea of the program being a simulation of a circuits lab so that the logic behind it can be more clearly explained. Consider the user of the program to be a student in an undergraduate circuits lab. This student wants to build a circuit using components from the lab's demonstrator, who has this library of components available. This library must contain components which are manufacturable, and we must have some way of enforcing this. The validation files can be seen as catalogues of manafacturable components from which the lab demonstrator must choose from to add to their library.

The base `Component` class contains two functions which are used by the derived classes to perform the validation checks on the components. The first is `load_validation_file` which takes in the name of the file, as well as the record to which to save it to. This record is something each derived class has, and is a vector of the `ValidationRecord` struct, whose data members are equivalent to each field in a line of the validation files (component name, maximum value, minimum value, tolerance, maximum frequency). `load_validation_file`also performs checks on the file for any blank lines or extra characters. The second function which `Component` has is `validate_component`. This function takes values from a user who wishes to intialise a new component or circuit, and checks that they match the record for the corresponding class. `validate_component` is

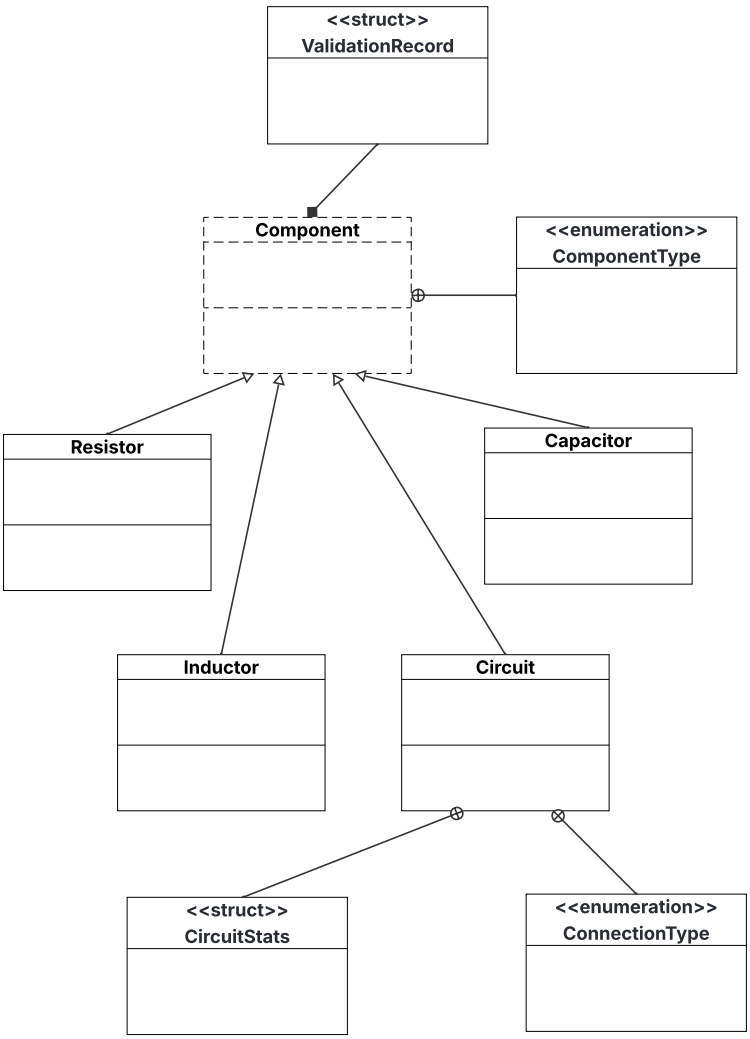


Figure 1: UML diagram of program, depicting all the classes in the program. Access online via `https://lucid.app/lucidchart/df2df27e-50a9-475c-90f7-234e81378d8d/view`. Enumerator classes and structs represented as nested classes where they are defined in the same implementation file.

used within all constructors and setters of the derived classes, apart from the `Circuit` class which does not have any validation checks. That being said, the `Circuit` class does use `validate_component` if a component is added to a circuit, or its frequency if set via a setter, to check that the frequency being set does not violate the maximum frequency of a component inside a circuit. This is because when a component is added to a circuit, its frequency is set to match the circuit's frequency. In the analogy, it can be thought that the lab demonstrator is keeping the components at some set frequency in their library, which is changed when they are added to a circuit.

Within the validation function, it was useful to use a lambda function because checks for extra characters and blank lines were the same for all fields in the file. Listing 1 shows the lambda function used to perform checks for extra characters in each field in one line of the file. Line 1 is where the lambda function is declared, and the entries in the square brackets is where its use becomes really beneficial - `line` and `comonent_type_str` are passed into the lambda function *implicitly* without needing to be passed as a parameter. This means the function can take the current line which is being read, and the component type (e.g. resistor) from the lines above the lambda function (which is already inside the validation function). These are useful in producing custom error messages on lines 8 and 9, making it easier for the user to understand where something exactly has gone wrong - it prints out the component name so we know which validation file the problem has occurred, and it prints out the whole line so we know where to search for the problem in that file.

```
1   auto parse_double = [&line, &component_type_str](const string& field, const string& field_name, double& output)
2   {
3     std::stringstream field_ss(field);
4     string extra;
5
6     if(!(field_ss >> output) || (field_ss>>extra))
7     {
8       std::cerr<<component_type_str<<" validation file error: Extra or invalid characters in "<<
9         field_name<<" in the line {"<<line<<"}. Line will not contribute."<<std::endl;
10
11      return false; //parse failed
12    }
13
14    return true;
```

Listing 1: Lambda function in validate component function (C++)

If the student wants to use one of the components from the library of the lab demonstrator, they cannot take it directly away from the lab demonstrator, as they are the owner of that specific component. This follows from the fact that the the library which is set up in main is a vector of unique pointers. There was a consideration to make it a vector of shared pointers, in which case the student could "share" the component with the lab demonstrator, and put it in the circuit (which would also share ownership). This came with a big drawback, which was that if a component is added to a circuit, its frequency would be updated, and since its shared, this would affect the component in all places it was used. Listing 2 shows the case where the component is shared amongst two circuits, and when the frequency of the second one is set to a different value of the first, and then

we try to access the info of the first circuit, the component's frequency will not match the circuit's frequency, giving the incorrect calculation of impedance for that circuit.

To avoid this, the student must make a copy of the component from the library, at which point they can add it to their circuit. Since unique pointers are being used, this requires the introduction of a `clone()` function which all of the `Component`-derived classes are given. It performs a deep copy of the component, akin to how a copy constructor works. This also came to be very useful for the copy constructor and assignment operator in the Circuit class, which containing a vector of unique pointers must obide by at least the rule of 3. Since we do want the user to also be able to move the circuit from one place to another, we implement the rule of 5, but set the move copy constructor and assignment operator to be default, as it is correctly handled by the unique pointer's `std::move` function.

```
1 circuit1.setFrequency(50);
2 circuit2.setFrequency(1000);  // overwrites the component's
     frequency
3 circuit1.info();              // component now thinks f=1000
```

Listing 2: Shared pointers don't work well for library of components

If the lab demonstrator lets the student take one component which they own, then it would no longer be in the library and no one else in the lab could use it, because it is stored using a unique pointer - there can only be one owner. The student must make a copy to use it in a circuit .In that sense it is not logical for identical components to be stored twice in the library, since infinitely many copies can be made of it. The lab demonstrator holds an "example" which must be copied to use. This idea is further supported by the fact that when added to the library, the components do not have IDs - controlled by setting a boolean parameter in the derived classes' constructors to `false`. Note this is not a parameter which is a data member of the derived classes - it lives in the constructor.

One must be careful about the copy semantics for the component classes when it comes to the IDs of each component. When the student "copies" a component from the library, what happens to the ID? Does it keep the same ID as the ID of the component in the library? Given that the components owned by the library do not have IDs, this would mean when they are copied to a circuit they would also not have IDs. To avoid this, the copy constructor manually increments the ID, so that if a component is copied, its ID is incremented. This is especially useful if the user wants to copy one component from one circuit and use it in another circuit. This component is not the same, because adding it to another circuit may change its frequency, and so it needs to be given its own ID. For the assignment copy operator, it is a bit different, because it starts with two already-existing components.

Consider listing 3, which has two resistors `r1` and `r2`, and say when they were added to a circuit they were given IDs `R1` and `R2`. If we assign `r1` to `r2`, then by default it would copy the ID of `r1` to `r2`. This doesn't make much sense, since then there would be two components with the same ID. It also doesn't make much sense for `r2` to be given a new ID, say `R3`, because it is the *same* component, just that it now has the same properties as `r1`. So the assignment operator is customised to copy everything but the ID, leaving `r2` with `R2` as an ID. The copy constructor doesn't suffer this issue since it doesn't have to deal with two components which already exist, and so it remains the most sensible to make the copy constructor increment the ID. Note that the rule of 0 is applied here, since although we have a customised copy constructor and assignment operator, this is

not because there are any non-class types (like pointers) in the class, and so the rule of 3 and 5 are not applicable.

```
Resistor r1(100, 1e2, 0.2, "Wirewound");
Resistor r2(2e4, 3, 0.05, "Carbon film");

r2=r1;
```

Listing 3: Using assignment operator for components

3. Results

Listing 4 shows a simplified version of the main file in this program. Line 1 creates the library of components using unique pointers, and then two components (Wirewound resistor and Ceramic capacitor) are added to the library. They are then both added to one circuit using the clone function, and the capacitor is added to another using the same clone function. These are both copies of the components from the library, so are stored in a different address and owned uniquely by the corresponding circuit. When they are added to the library, they are set to have no IDs (there is not much use in labelling them inside the library), but when they are added to the circuit, since the copy constructor needs to be used (via `clone()`), they are given an ID.

The final two lines produce outputs for the parallel circuit. `info()` gives the combined impedance of the circuit as well as its behaviour (capacitive or inductive). It also produces the number of components contained within the circuit via the `count_components()` function. This function makes use of the `CircuitStats` struct to store the number of occurrences of each component as one object. `count_components()` is a recursive function which deals with 2 levels of hierarchy (i.e. a circuit within a circuit) in order to not only produce the correct number of components, but also the correct number of nested series and parallel circuits.

```
vector<std::unique_ptr<Component>> library;
library.push_back(std::make_unique<Resistor>(100, 1e2, 0.2,
    "Wirewound", false));
library.push_back(std::make_unique<Capacitor>(1000, 5e-7,
    0.1, "Ceramic capacitor", false));

auto circuit_1 = std::make_unique<Circuit>(50,
    ConnectionType::Series);
circuit_1->add_component(library[0]->clone()); //Wirebound
    resistor
circuit_2->add_component(library[1]->clone()); //Ceramic
    capacitor

auto circuit_2 = std::make_unique<Circuit>(60,
    ConnectionType::Series);
circuit_2->add_component(library[1]->clone()); //Ceramic
    capacitor

auto combined_parallel = std::make_unique<Circuit>(55,
    ConnectionType::Parallel);
combined_parallel->add_component(std::move(circuit_1));
combined_parallel->add_component(std::move(circuit_2));
```

```
15
16 combined_parallel->info();
17 combined_parallel->component_info();
```

Listing 4: Example of program functionality

4. Discussion and conclusions

The program is effective in the scenario that has been described throughout this report, that of imagining a student in a lab, building a circuit with limited actions (creating series and parallel circuits one by one), and constrained by the lab demonstrator's library of available components. One disadvantage of the program is that it does not account for validation files which give unreasonable values. For example, the user could use their own validation file which has a negative frequency or, the tolerance could be a number not between 0 and 1. There is no protection against this in the code.

Another issue comes about if the user does not intialise circuits using a unique pointer, and instead just declares them as a `Circuit` object. When a circuit is moved to another circuit, it is added as a component to that circuit, and so must be converted to a unique pointer. This means the pointer to the circuit in its old address becomes null, but that original circuit object still exists and its info function could be called. This produces an output but with all values set to 0, which doesn't make much sense. One way around this is to use a boolean value to keep track of whether a circuit has been moved, and if it has, print out a different message over the normal info function message warning the user that it is a moved function. Also, once a function is moved into a circuit, it cannot be modified or accessed individually. A nice feature would be to let the user access the components in a circuit using their IDs, and then be able to call a setter or getter on a specific component.

References

[1] All About Circuits. Electric fields and capacitance. https://www.allaboutcircuits.com/textbook/direct-current/chpt-13/electric-fields-capacitance/. Accessed: 16 May 2026.

[2] cplusplus.com contributors. Difference between `dynamic_cast` and `static_cast`. https://cplusplus.com/forum/unices/132444/. Accessed: 16 May 2026.

[3] Digi-Key Electronics. Capacitor kits. https://www.digikey.co.uk/en/products/filter/capacitor-kits/651. Accessed: 16 May 2026.

[4] EEPower. Resistor materials. https://eepower.com/resistor-guide/resistor-materials/. Accessed: 16 May 2026.

[5] EEPower. Types of capacitors. https://eepower.com/capacitor-guide/types. Accessed: 16 May 2026.

[6] Electronics For You. Circuit design calculator using C. https://www.electronicsforu.com/electronics-projects/software-projects-ideas/circuit-design-calculator-using-c. Accessed: 16 May 2026.

[7] Electronics Tutorials. Resistor in AC circuits. https://www.electronics-tutorials.ws/resistor/res_8.html. Accessed: 16 May 2026.

[8] Glenn Elert. Inductance. https://physics.info/inductance/, 2025. Accessed: 16 May 2026.

[9] Andreas Fertig. How to use a `constexpr std::string`. https://andreasfertig.com/blog/2025/01/how-to-use-a-constexpr-stdstring/, Jan 2025. Accessed: 16 May 2026.

[10] GeeksforGeeks. Complex numbers in C++ | set 1. https://www.geeksforgeeks.org/cpp/complex-numbers-c-set-1/, Jan 2017. Accessed: 16 May 2026.

[11] GeeksforGeeks. `push_back()` vs `emplace_back()` in C++ STL vectors. https://www.geeksforgeeks.org/cpp/push_back-vs-emplace_back-in-cpp-stl-vectors/, Mar 2021. Accessed: 16 May 2026.

[12] Interactive Mathematics. 9. impedance and phase angle. https://www.intmath.com/complex-numbers/9-impedance-phase-angle.php. Accessed: 16 May 2026.

[13] Iowa State University Center for Nondestructive Evaluation. Impedance. https://www.nde-ed.org/Physics/Electricity/impedance.xhtml. Accessed: 16 May 2026.

[14] Isaac Physics. Electrical impedance. https://isaacscience.org/concepts/cp_electrical_impedance. Accessed: 16 May 2026.

[15] Samuel J. Ling, William Moebs, and Jeff Sanny. *University Physics Volume 2*. OpenStax, Houston, Texas, 2016.

[16] Microsoft. Math constants. https://learn.microsoft.com/en-us/cpp/c-runtime-library/math-constants?view=msvc-170. Accessed: 16 May 2026.

[17] Microsoft. Type double. https://learn.microsoft.com/en-us/cpp/c-language/type-double?view=msvc-170. Accessed: 16 May 2026.

[18] MTI Instruments / Vitrek. Capacitor characteristics. https://vitrek.com/mti-instruments/knowledge-center/capacitor-characteristics/, Aug 2023. Accessed: 16 May 2026.

[19] Rod Nave. Electric field. http://hyperphysics.phy-astr.gsu.edu/hbase/electric/elefie.html. Accessed: 16 May 2026.

[20] Nick0412. Architecture of circuit solver. https://gist.github.com/Nick0412/0b9a0ba8848cb5225d5bd663dad8fd77. Accessed: 16 May 2026.

[21] Arthur O'Dwyer. Just how constexpr is C++20's std::string? https://quuxplusone.github.io/blog/2023/09/08/constexpr-string-firewall/, Sep 2023. Accessed: 16 May 2026.

[22] Madhawa Polkotuwa. Understanding static and dynamic casting in C++. https://madhawapolkotuwa.medium.com/understanding-static-and-dynamic-casting-in-c-b73cf3f92ec2, Jan 2025. Accessed: 16 May 2026.

[23] RS Components. Aluminium electrolytic capacitors datasheet. https://docs.rs-online.com/49da/0900766b80fa3a92.pdf. Accessed: 16 May 2026.

[24] Scribd. Kirchhoff's law in C program. https://es.scribd.com/document/128210104/kirchoff-law-in-c-program. Accessed: 16 May 2026.

[25] Stack Overflow contributors. Answer: Why does std::vector need the move constructor to be noexcept? https://stackoverflow.com/a/20521414. Accessed: 16 May 2026.

[26] Stack Overflow contributors. C++ design for circuit solver. https://stackoverflow.com/questions/59160172/c-design-for-circuit-solver. Accessed: 16 May 2026.

[27] Paul Peter Urone and Roger Hinrichs. *College Physics 2e.* OpenStax, Houston, Texas, 2022.

[28] Paul Peter Urone, Roger Hinrichs, Kim Dirks, and Manjula Sharma. *College Physics 1e.* LibreTexts, 2021.

[29] Wikipedia contributors. Electrical reactance. https://en.wikipedia.org/wiki/Electrical_reactance. Accessed: 16 May 2026.

[30] Wikipedia contributors. Nodal analysis. https://en.wikipedia.org/wiki/Nodal_analysis#Matrix_form_for_the_node-voltage_equation. Accessed: 16 May 2026.